

CTE ASSIGNMENT

SQL Questions Q201–Q250

```
-- =====
-- SECTION 5 – CTE
-- =====

-- Q201. Display all customers using a CTE.
WITH customer_data AS (
    SELECT customer_id, customer_name, city
    FROM customers
)
SELECT *
FROM customer_data;

-- Q202. Display customers from Mumbai using a CTE.
WITH customer_data AS (
    SELECT customer_id, customer_name, city
    FROM customers
)
SELECT *
FROM customer_data
WHERE city = 'Mumbai';

-- Q203. Display all products with price greater than 10000 using a CTE.
WITH product_data AS (
    SELECT product_id, product_name, price
    FROM products
)
SELECT *
FROM product_data
WHERE price > 10000;

-- Q204. Display products with stock greater than 100 using a CTE.
WITH product_data AS (
    SELECT product_id, product_name, stock
    FROM products
)
SELECT *
FROM product_data
WHERE stock > 100;

-- Q205. Find the average product price using a CTE.
WITH product_prices AS (
    SELECT price
    FROM products
)
SELECT AVG(price) AS average_price
FROM product_prices;

-- Q206. Find the highest product price using a CTE.
WITH product_prices AS (
    SELECT price
    FROM products
)
SELECT MAX(price) AS highest_price
FROM product_prices;

-- Q207. Find the lowest product price using a CTE.
WITH product_prices AS (
    SELECT price
    FROM products
)
SELECT MIN(price) AS lowest_price
FROM product_prices;

-- Q208. Find the total stock of all products using a CTE.
WITH product_stock AS (
    SELECT stock
    FROM products
)
SELECT SUM(stock) AS total_stock
FROM product_stock;
-- Q209. Count customers in each city using a CTE.
WITH city_customers AS (
```

```

        SELECT city, COUNT(*) AS total_customers
        FROM customers
        GROUP BY city
    )
    SELECT *
    FROM city_customers;

-- Q210. Display cities having more than 5 customers using a CTE.
WITH city_customers AS (
    SELECT city, COUNT(*) AS total_customers
    FROM customers
    GROUP BY city
)
SELECT *
FROM city_customers
WHERE total_customers > 5;

-- Q211. Find total orders for each customer using a CTE.
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_orders;

-- Q212. Find total amount spent by each customer using a CTE.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_spending;

-- Q213. Display customers whose total spending is greater than 50000.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_spending
WHERE total_spent > 50000;

-- Q214. Find average order amount for each customer.
WITH customer_orders AS (
    SELECT customer_id, AVG(total_amount) AS average_order_amount
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_orders;

-- Q215. Find the highest order amount for each customer.
WITH customer_orders AS (
    SELECT customer_id, MAX(total_amount) AS highest_order
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_orders;

-- Q216. Find the total quantity sold for each product.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT *
FROM product_sales;

-- Q217. Display products whose total quantity sold is greater than 5.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id

```

```

)
SELECT *
FROM product_sales
WHERE total_quantity > 5;

-- Q218. Find the average quantity ordered for each product.
WITH product_quantity AS (
    SELECT product_id, AVG(quantity) AS average_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT *
FROM product_quantity;

-- Q219. Find the total discount for each product.
WITH product_discounts AS (
    SELECT product_id, SUM(discount) AS total_discount
    FROM order_items
    GROUP BY product_id
)
SELECT *
FROM product_discounts;

-- Q220. Display order statuses and their order counts.
WITH order_status_count AS (
    SELECT order_status, COUNT(*) AS total_orders
    FROM orders
    GROUP BY order_status
)
SELECT *
FROM order_status_count;

-- Q221. Display order statuses having more than 10 orders.
WITH order_status_count AS (
    SELECT order_status, COUNT(*) AS total_orders
    FROM orders
    GROUP BY order_status
)
SELECT *
FROM order_status_count
WHERE total_orders > 10;

-- Q222. Find total order amount for each order status.
WITH status_amount AS (
    SELECT order_status, SUM(total_amount) AS total_amount
    FROM orders
    GROUP BY order_status
)
SELECT *
FROM status_amount;

-- Q223. Display order statuses having total amount greater than 100000.
WITH status_amount AS (
    SELECT order_status, SUM(total_amount) AS total_amount
    FROM orders
    GROUP BY order_status
)
SELECT *
FROM status_amount
WHERE total_amount > 100000;

-- Q224. Find average price for each category.
WITH category_prices AS (
    SELECT category_id, AVG(price) AS average_price
    FROM products
    GROUP BY category_id
)
SELECT *
FROM category_prices;

-- Q225. Find categories having average product price greater than 10000.
WITH category_prices AS (
    SELECT category_id, AVG(price) AS average_price
    FROM products
    GROUP BY category_id
)
SELECT *

```

```

FROM category_prices
WHERE average_price > 10000;

-- Q226. Find total stock for each category.
WITH category_stock AS (
    SELECT category_id, SUM(stock) AS total_stock
    FROM products
    GROUP BY category_id
)
SELECT *
FROM category_stock;

-- Q227. Find brands having more than 2 products.
WITH brand_products AS (
    SELECT brand_id, COUNT(*) AS total_products
    FROM products
    GROUP BY brand_id
)
SELECT *
FROM brand_products
WHERE total_products > 2;

-- Q228. Find average product price for each brand.
WITH brand_prices AS (
    SELECT brand_id, AVG(price) AS average_price
    FROM products
    GROUP BY brand_id
)
SELECT *
FROM brand_prices;

-- Q229. Find brands having total stock greater than 500.
WITH brand_stock AS (
    SELECT brand_id, SUM(stock) AS total_stock
    FROM products
    GROUP BY brand_id
)
SELECT *
FROM brand_stock
WHERE total_stock > 500;

-- Q230. Display customer names with their total order amounts using CTE and JOIN.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, cs.total_spent
FROM customers c
INNER JOIN customer_spending cs
ON c.customer_id = cs.customer_id;

-- Q231. Display customer names with their total number of orders.
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, co.total_orders
FROM customers c
INNER JOIN customer_orders co
ON c.customer_id = co.customer_id;

-- Q232. Display product names with their total quantity sold.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT p.product_name, ps.total_quantity
FROM products p
INNER JOIN product_sales ps
ON p.product_id = ps.product_id;

-- Q233. Display product names with their total discount.
WITH product_discounts AS (
    SELECT product_id, SUM(discount) AS total_discount

```

```

        FROM order_items
        GROUP BY product_id
    )
SELECT p.product_name, pd.total_discount
FROM products p
INNER JOIN product_discounts pd
ON p.product_id = pd.product_id;

-- Q234. Display category names with the number of products.
WITH category_products AS (
    SELECT category_id, COUNT(*) AS total_products
    FROM products
    GROUP BY category_id
)
SELECT c.category_name, cp.total_products
FROM categories c
INNER JOIN category_products cp
ON c.category_id = cp.category_id;

-- Q235. Display brand names with the number of products.
WITH brand_products AS (
    SELECT brand_id, COUNT(*) AS total_products
    FROM products
    GROUP BY brand_id
)
SELECT b.brand_name, bp.total_products
FROM brands b
INNER JOIN brand_products bp
ON b.brand_id = bp.brand_id;

-- Q236. Display customer names and their total spending above 50000.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
    HAVING SUM(total_amount) > 50000
)
SELECT c.customer_name, cs.total_spent
FROM customers c
INNER JOIN customer_spending cs
ON c.customer_id = cs.customer_id;

-- Q237. Display product names whose total quantity sold is greater than 5.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
    HAVING SUM(quantity) > 5
)
SELECT p.product_name, ps.total_quantity
FROM products p
INNER JOIN product_sales ps
ON p.product_id = ps.product_id;

-- Q238. Display category names having more than 2 products.
WITH category_products AS (
    SELECT category_id, COUNT(*) AS total_products
    FROM products
    GROUP BY category_id
    HAVING COUNT(*) > 2
)
SELECT c.category_name, cp.total_products
FROM categories c
INNER JOIN category_products cp
ON c.category_id = cp.category_id;

-- Q239. Display brand names having more than 2 products.
WITH brand_products AS (
    SELECT brand_id, COUNT(*) AS total_products
    FROM products
    GROUP BY brand_id
    HAVING COUNT(*) > 2
)
SELECT b.brand_name, bp.total_products
FROM brands b
INNER JOIN brand_products bp
ON b.brand_id = bp.brand_id;
-- Q240. Display order IDs with their payment methods using CTE and JOIN.
WITH payment_data AS (

```

```

        SELECT order_id, payment_method, payment_status
        FROM payments
    )
    SELECT o.order_id, pd.payment_method, pd.payment_status
    FROM orders o
    INNER JOIN payment_data pd
    ON o.order_id = pd.order_id;

-- Q241. Display order IDs with their shipping status using CTE and JOIN.
WITH shipping_data AS (
    SELECT shipping_id, shipping_status, city
    FROM shipping
)
SELECT o.order_id, sd.shipping_status, sd.city
FROM orders o
INNER JOIN shipping_data sd
ON o.shipping_id = sd.shipping_id;

-- Q242. Display customer names with their shipping cities.
WITH order_shipping AS (
    SELECT customer_id, shipping_id
    FROM orders
)
SELECT c.customer_name, s.city
FROM order_shipping os
INNER JOIN customers c
ON os.customer_id = c.customer_id
INNER JOIN shipping s
ON os.shipping_id = s.shipping_id;

-- Q243. Display product names with category names using CTE and JOIN.
WITH product_categories AS (
    SELECT product_id, product_name, category_id
    FROM products
)
SELECT pc.product_name, c.category_name
FROM product_categories pc
INNER JOIN categories c
ON pc.category_id = c.category_id;

-- Q244. Display product names with brand names using CTE and JOIN.
WITH product_brands AS (
    SELECT product_id, product_name, brand_id
    FROM products
)
SELECT pb.product_name, b.brand_name
FROM product_brands pb
INNER JOIN brands b
ON pb.brand_id = b.brand_id;

-- Q245. Use two CTEs to display customer names and total orders.
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
),
customer_data AS (
    SELECT customer_id, customer_name
    FROM customers
)
SELECT cd.customer_name, co.total_orders
FROM customer_data cd
INNER JOIN customer_orders co
ON cd.customer_id = co.customer_id;

-- Q246. Use two CTEs to display customer names and total spending.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
),
customer_data AS (
    SELECT customer_id, customer_name
    FROM customers
)
SELECT cd.customer_name, cs.total_spent
FROM customer_data cd
INNER JOIN customer_spending cs
ON cd.customer_id = cs.customer_id;

```

```

-- Q247. Use two CTEs to display product names and total quantity sold.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
),
product_data AS (
    SELECT product_id, product_name
    FROM products
)
SELECT pd.product_name, ps.total_quantity
FROM product_data pd
INNER JOIN product_sales ps
ON pd.product_id = ps.product_id;

-- Q248. Use two CTEs to display category names and average product price.
WITH category_prices AS (
    SELECT category_id, AVG(price) AS average_price
    FROM products
    GROUP BY category_id
),
category_data AS (
    SELECT category_id, category_name
    FROM categories
)
SELECT cd.category_name, cp.average_price
FROM category_data cd
INNER JOIN category_prices cp
ON cd.category_id = cp.category_id;

-- Q249. Use multiple CTEs to display customer name, total orders and total spending.
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
),
customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
),
customer_data AS (
    SELECT customer_id, customer_name
    FROM customers
)
SELECT cd.customer_name,
       co.total_orders,
       cs.total_spent
FROM customer_data cd
INNER JOIN customer_orders co
ON cd.customer_id = co.customer_id
INNER JOIN customer_spending cs
ON cd.customer_id = cs.customer_id;

-- Q250. Use multiple CTEs to display product name, category name, brand name and total quantity sold.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
),
product_data AS (
    SELECT product_id, product_name, category_id, brand_id
    FROM products
),
category_data AS (
    SELECT category_id, category_name
    FROM categories
),
brand_data AS (
    SELECT brand_id, brand_name
    FROM brands
)
SELECT pd.product_name,
       cd.category_name,
       bd.brand_name,
       ps.total_quantity
FROM product_data pd
INNER JOIN product_sales ps
ON pd.product_id = ps.product_id
INNER JOIN category_data cd
ON pd.category_id = cd.category_id
INNER JOIN brand_data bd
ON pd.brand_id = bd.brand_id;

```

```
ON pd.brand_id = bd.brand_id;
```